



GENERALITAT
VALENCIANA

Conselleria d'Educació,
Universitats i Ocupació

ceedcv

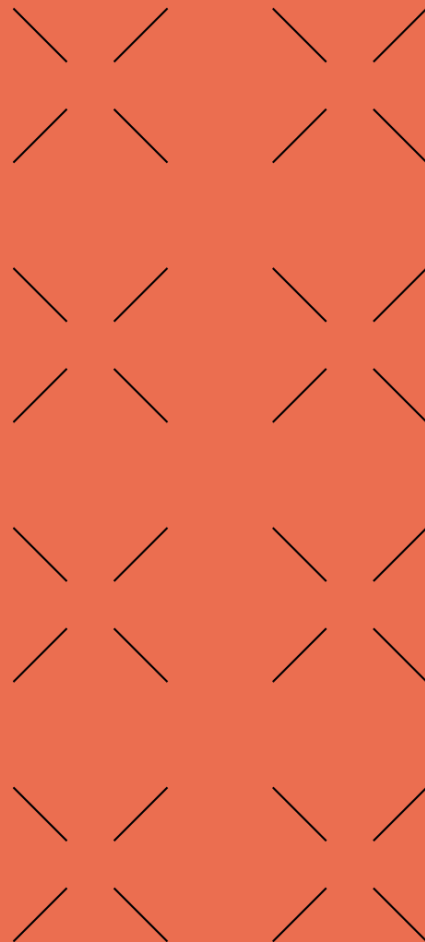
CENTRE ESPECÍFIC
D'EDUCACIÓ A DISTÀNCIA DE
LA COMUNITAT VALENCIANA

Unitat 04 — Estructures de control i excepcions

Bloc 02

1r CFGS DAW · Programació

Sergi García Barea



Bucle for

La idea en una frase

“

`for` és un bucle amb **comptador de sèrie**: declara la variable, posa la condició i l'actualitza en la mateixa línia, ideal per a "repeteix N vegades".

El while repeteix "mentres passe alguna cosa". El for repeteix "un nombre exacte de vegades". És el bucle favorit per a recórrer coses i el que més usaràs en tota la teua carrera.

L'anatomia del for

Voltes.java

Java

```
for (int i = 1; i ≤ 5; i++) {  
    System.out.println("Volta " + i);  
}
```

- 1 Inicialització:** `int i = 1` — una sola vegada, en entrar
- 2 Condició:** `i ≤ 5` — abans de cada volta
- 3 Bloc:** s'executa si la condició és true
- 4 Actualització:** `i++` — al final de cada volta



Una coma mal posada en el for és l'error d'examen més comú: **tres parts, dos punts i coma, zero comes.**

Els tres bucles són el mateix acudit

TresBucles.java

Java

```
int i = 1;                // while: les 3 parts repartides
while (i ≤ 5) {
    System.out.println("Volta " + i);
    i++;
}
for (int j = 1; j ≤ 5; j++) { // for: les 3 parts en una línia
    System.out.println("Volta " + j);
}
```

✓ Els dos imprimixen el mateix. El for guanya perquè junta les tres parts del control en una línia: és més difícil oblidar el `i++` (adéu, bucles infinits per descuit). **Regla:** saps quantes voltes → for; no ho saps → while.

Bucles anidats: la graella d'exercicis

Graella.java

Java

```
for (int fila = 1; fila ≤ 3; fila++) {  
    for (int col = 1; col ≤ 4; col++) {  
        System.out.print("* ");  
    }  
    System.out.println();  
}
```

```
* * * *  
* * * *  
* * * *
```

- 1 L'exterior controla les **files** (3)
- 2 L'interior controla les **columnes** (4)

 Cada nivell d'anidament **multiplica** les voltes: $1000 \times 1000 = 1.000.000$ d'iteracions. Els bucles anidats són la fàbrica de programes lents.

Exemple guiat: la taula de multiplicar

TaulaMultiplicar.java

Java

```
for (int i = 1; i ≤ 10; i++) {  
    System.out.println("7 x " + i + " = " + (7 * i));  
}
```

7 x 1 = 7

7 x 2 = 14

7 x 3 = 21

...

El triangular: l'interior depèn de l'exterior

Triangle.java

Java

```
for (int fila = 1; fila ≤ 4; fila++) {  
    for (int ast = 1; ast ≤ fila; ast++) {  
        System.out.print("*");  
    }  
    System.out.println();  
}
```

```
*  
**  
***  
****
```



La condició de l'interior és `ast ≤ fila` : cada fila imprimix tants asteriscs com número de fila. **10 asteriscs** en total (1+2+3+4). Que l'interior depenga de l'exterior és el cor dels bucles anidats.

Mini-chequeig: for



Respon en 30 segons:

1. Quines són les tres parts del for?
2. Quantes vegades s'executa la inicialització?
3. Què imprimix `for (int i = 0; i < 5; i++)` ? ¿5 o 4 voltes?
4. En un bucle anidat, què fa l'interior per cada volta de l'exterior?



Respostes:

- **Inicialització, condició i actualització**, separades per ;
- **Una sola vegada**, en entrar en el bucle
- **5 voltes**: amb i valent 0, 1, 2, 3 i 4 (compte amb l'off-by-one!)
- S'executa **complet** (totes les seues voltes) per cada volta de l'exterior

break: el botón de parada

La idea en una frase

“

`break` apaga el bucle sencer i `continue` es salta només la volta actual; amb **etiquetes** pots decidir a quin bucle anidat afecten.

Ja saps repetir. Ara toca aprendre a **eixir amb estil**: interrompre, saltar i dirigir-te a un bucle concret quan n'hi ha diversos.

break: acaba el bucle immediatament

Parada.java

Java

```
for (int i = 1; i ≤ 10; i++) {  
    if (i = 5) {  
        break;  
    }  
    System.out.println(i);  
}
```

1

2

3

4

 Tan bon punt i val 5, break talla: les voltes 6 a 10 no ocorren mai. Perfecte per a "troba alguna cosa i para de buscar".

continue: el botó de saltar

Parells.java

Java

```
int suma = 0;
for (int i = 1; i ≤ 10; i++) {
    if (i % 2 ≠ 0) {
        continue; // senars: no compten
    }
    suma += i;
}
System.out.println("Parells: " + suma);
```

```
$ java Parells
Parells: 30
```

⚠ En un **while**, si poses el **continue** **abans** d'actualitzar la variable, l'actualització es salta... i el bucle no avança: bug infinit assegurat. En un **for** no passa res: l'actualització és a la capçalera.

Etiquetes: el GPS dels bucles anidats

Exterior.java

Java

```
exterior:
for (int i = 1; i ≤ 3; i++) {
    for (int j = 1; j ≤ 3; j++) {
        if (i * j ≥ 6) {
            break exterior;
        }
        System.out.println(i + " x " + j);
    }
}
```

```
1 x 1
1 x 2
1 x 3
2 x 1
2 x 2
```



Un break sol afecta **només el bucle més intern**. `break exterior;` ix dels **dos bucles** ahora.



Les etiquetes són legals però **poc usades**: quasi sempre es pot redissenyar amb una variable booleana. Usa-les amb moderació: el teu company de projecte t'ho agrairà.

Exemple guiat: el detector de nombres primers

EsPrimer.java

Java

```
int numero = 29;
boolean esPrimer = true;
for (int divisor = 2; divisor < numero; divisor++) {
    if (numero % divisor == 0) {
        esPrimer = false;
        break; // trobat divisor: para de buscar
    }
}
System.out.println(numero + " és primer? " + esPrimer);
```

```
$ java EsPrimer
29 és primer? true
```

Mini-chequeig: break i continue



Respon sense mirar:

1. Quina és la diferència entre break i continue en una frase?
2. A què afecten per defecte en bucles anidats?
3. Per a què servix una etiqueta?
4. Per què és perillós el continue en un while si va abans de l'actualització?



Respostes:

- break **acaba** el bucle; continue **salta només la volta actual**
- Al bucle **més intern**
- Perquè un break o continue afecte un bucle exterior concret (`break etiqueta;`)
- Perquè l'actualització es salta i el bucle **no avança**: condició true per sempre

Excepcions bàsiques

La idea en una frase

“

Una excepció és un **avís que alguna cosa ha eixit malament**: Java el llança com un objecte que hereta de `Throwable`, i el teu programa pot estar o no preparat per a atrapar-lo.

T'ha passat que un programa es "cau" amb un munt de text roig? Eixe text és una excepció. En comptes de morir en silenci, Java crida amb tot el detall. Aprendre a llegir eixos crits és aprendre a depurar.

El crash: la teua primera excepció

Explosio.java

Java

```
int[] numeros = { 1, 2, 3 };  
System.out.println(numeros[5]); // no existix!
```

```
Exception in thread "main"  
java.lang.ArrayIndexOutOfBoundsException:  
Index 5 out of bounds for length 3  
at Explosio.main(Explosio.java:5)
```



Este text és or: et diu **què** (l'excepció), **on** (línia 5) i en **quin mètode**. Llegir-lo bé resol la mitat dels teus problemes.

La família Throwable: l'arbre genealògic

```
Object
├── Throwable
│   ├── Error
│   └── Exception
│       └── RuntimeException
```

⚠ **Error:** problemes greus de la JVM (memòria esgotada). No els provoques tu i no has d'intentar atrapar-los.

- 1 Throwable:** la classe arrel de tot el que es llança
- 2 Exception:** fallades del programa. Ací viu el 99% de la teua vida
- 3 RuntimeException:** es llança en temps d'execució i **no estàs obligat** a capturar-la

Les excepcions més comunes: la guia de camp

Excepció	Quan apareix	Frase típica
ArithmeticException	Dividir entre 0	"Dividir entre zero, quin valent"
ArrayIndexOutOfBoundsException	Índex fora de l'array	"Eixe buit no existix"
NullPointerException	Cridar alguna cosa null	"El clàssic absolut"
NumberFormatException	Text que no és nombre	"Convertir 'Hola' en nombre, no"
InputMismatchException	Scanner rep el tipus equivocacat	"Text on anava un nombre"



La **NullPointerException** és, amb diferència, l'excepció més comuna de la història de Java: apareix en tocar un objecte que val `null`.

Mini-chequeig: excepcions



Quina excepció llançaria cada línia?

1. `int a = 5 / 0;`
2. `dies[3]` amb un array de 3 elements
3. `text.toUpperCase()` amb `text = null`
4. `Integer.parseInt("quaranta-dos")`



Respostes:

- **ArithmeticException** (divisió entre zero)
- **ArrayIndexOutOfBoundsException** (un array de 3 s'indexa 0, 1, 2)
- **NullPointerException** (text és null)
- **NumberFormatException** (eixe text no és un nombre)

Resum del bloc 02



for

Inici; condició; avanç en una línia; anidats = multiplicar voltes



break / continue

Apaga el bucle vs salta la volta; etiquetes per a anidats



Excepcions

Throwable → Error / Exception
→ RuntimeException; llig el stack trace



Bloc 03: **try/catch/finally** per a blindar el programa, **throw** i excepcions pròpies, i el repàs final.

